

World-Building

Application d'archivage et de visualisation de données

Projet réalisé dans le cadre de la présentation au
Titre Professionnel -DWWM-
Développeur Web et Web Mobile

Présenté par
Blanc Dylan

Organisme de Formation
LaPlateforme Toulon 2025/2026

SOMMAIRE

I	Sommaire	2
II	Introduction	4
III	Résumé du projet	4
IV	Présentation	5
	IV.a Users story	5
	IV.b M.V.P	7
	IV.c Evolution	8
V	Conception	9
	V.a Wireframe	9
	V.b Routes	12
	V.c M.C.D	14
	V.d M.L.D	15

VI	Dictionnaire de donnée	16
VII	Environnement et Architecture	21
	VII.a Environnement de Développement	21
	VII.b Choix Technique	21
	VII.c Workflow	25
VIII	Frontend	27
IX	Sécurité	31
	X.a Frontend	31
	X.a Backend	35
X	Déploiement	39
XI	Annexe	43

Introduction

L'idée de créer ce projet m'est venu de part mon intérêt pour le "WorldBuilding" ou plus globalement pour l'intérêt que j'ai à imaginer des histoires ou des scénarios, de ma passion pour la fiction et enfin de mon comportement d'écureuil à stocker des datas sur ce que je trouve intéressant dans ce contexte

dans ce but je ne trouvais pas de support me permettant de stocker du texte, images, audio, vidéo et de les catégoriser correctement, de faire des liaison/annotation... tout en étant le plus visuel et simple possible et ainsi l'idée de ce projet est née, dans ce but ce projet consiste à créer une application permettant le stockage et archivage de data en ligne sur un serveur permettant le partage des " Pages Utilisateurs " à la manière d'un fansite mais avec une couche créatrice plus permissive - Mais aussi permettre de créer des pages privé stocker en Local qu'il sera possible d'importer ou exporter

Résumé du Projet

"WorldBuilding" doit permettre aux utilisateurs de stocker et archiver des datas dans des pages utilisateurs personnel qu'ils pourront partager publiquement ou conserver privé grâce à un CMS le plus complet et permissif possible ainsi qu'à un système WYSIWYG permettant la création et modification de pages utilisateurs en profondeur sans que le potentiel créatif soit bridé par des limites arbitraire

Presentation

Users Story

v1

En tant que : VISITEUR Je Souhaite : pouvoir CREER UN COMPTE et me CONNECTER facilement afin de gérer mes "pages" favoris, mes filtres favoris et même créer mes propres pages 2	En tant que : UTILISATEUR Je Souhaite : pouvoir mettre en FAVORIS les catégories qui m'intéresse et pouvoir rapidement SUIVRE l'actualité des "blogs" qui m'intéresse 3	En tant que : ADMINISTRATEUR Je Souhaite : être capable de visualiser la liste des utilisateurs, des blogs, des filtres et toutes statistique relatif a mon site, facilement et rapidement 5
En tant que : VISITEUR Je Souhaite : Visualiser la listes des blogs et les FILTERER selon les catégories qui m'intéresse 6	En tant que : UTILISATEUR Je Souhaite : pouvoir CREER UN "BLOG" selon les thèmes qui m'intéresse en étant le plus libre possible dans sa conception, textes, images, liens..... et de l'éditer si neccesaire 10	En tant que : ADMINISTRATEUR Je Souhaite : être capable de modérer mon site, de bannir ou suspendre les utilisateurs, de supprimer des textes/images/liens....de supprimer des "blog" individuellement ou en cascade 4
En tant que : VISITEUR Je Souhaite : pouvoir mettre en mode "SOMBRE" ou "CLAIR" le site pour le confort de mes yeux 2	En tant que : UTILISATEUR Je Souhaite : avoir des presets de configuration comme un Journal, une Galerie, une Mosaïque etc etc... lors de ma création de "blog" afin de faciliter sa création 8	En tant que : ADMINISTRATEUR Je Souhaite : être capable de créer de nouvelles catégories de filtres facilement ou de modifier les filtres existant 3
	En tant que : UTILISATEUR Je Souhaite : avoir la possibilité de rendre public ou privé mes pages personnel ainsi que leurs informations 4	

v2

En tant que : VISITEUR Je Souhaite : pouvoir choisir le theme de l'interface utilisateur ainsi que choisir la police d'écriture 3	En tant que : UTILISATEUR Je Souhaite : avoir accès à un "marketplace" me permettant de partager ou de choisir des presets de fabrication de page personnel 6	En tant que : ADMINISTRATEUR Je Souhaite : avoir la possibilité de modérer le contenu du marketplace, de supprimer le contenu indésirable ou d'inclure un "preset populaire" dans la selection "de base" du site 4
En tant que : VISITEUR Je Souhaite : pouvoir modifier l'affichage des "cards" sur la page d'accueil parmis un pannel de choix 3	En tant que : UTILISATEUR Je Souhaite : être capable de discuter avec les autres utilisateurs dans un format "forum" sous mes pages ou celles des autres ainsi que pouvoir modérée mes pages personnel 4	En tant que : ADMINISTRATEUR Je Souhaite : voir l'historique des messages utilisateurs, s'ils ont était "report" ainsi que le nombres de fois, avoir une liste d'utilisateur tros souvent report etc etc et avoir la possibilité de nettoyer les messages 4
En tant que : VISITEUR Je Souhaite : pouvoir voir les messages des utilisateurs sur les "pages" des contenu qui m'intéresse 2	En tant que : UTILISATEUR Je Souhaite : être capable de changer la police d'écriture global de mes pages ainsi que des portion de texte spécifique 3	En tant que : ADMINISTRATEUR Je Souhaite : avoir une liste des police d'écriture utiliser, leur popularité, la possibilité de les inclure dans le site "de base" de les interdire/supprimer 5
	En tant que : UTILISATEUR Je Souhaite : dessiner a l'aide de mon styilet sur une page blanche et l'intégrrer dans mes pages personnel 7	

M.V.P - Minimum Viable Product

le MVP concerne le gros du travail et ce qui définit mon application, ici la création de page utilisateurs et un CMS complexe WYSIWYG avec intégration d'un système de filtre avancée

MVP

Minimum Viable Product

Conception :

- Wireframe/Maquette de base
- MCD MLD MPD selon la methode Merise
- Users story
- Cahier des charges

Production :

- Dockerisation du projet
- stack :

Php Backend, Vuejs(Nuxtjs) Frontend

Rendu :

- Système d'authentification
- Authentification via Google Auth
- Système de filtre et recherche avancée
- Filtres favoris
- Création de page "blog" personnalisable + prévisualisation du résultat responsive
- Personnalisation WYSIWYG drag & drop pour les pages de blog
- Intégration WYSIWYG "style forum"
- Affichage des "blogs" en page d'accueil
- "Suivre" pour les pages de "blogs"
- Theme Sombre/Clair

Evolution

Après avoir défini les fonctionnalités prioritaires à mon projet ici je défini ce qui n'est pas nécessaire au fonctionnement du site et qui nécessiterait trop de temps comme par exemple l'intégration d'un Marketplace utilisateur ou l'intégration du stylet et dessin de l'utilisateur, l'intégration d'un plugin navigateur permettant une plus grande simplicité d'interaction entre les données de l'utilisateur et l'application

V2

Rendu :

- Intégration WYSIWYG "clic droit"
- Themes graphique personnalisable
- Police d'écriture personnel et global Personnalisable
- Plusieurs View pour l'affichage de la page d'accueil
- Marketplace de rendu WYSIWYG prédéfini par d'autres utilisateurs (Galerie, Journal etc etc...)
- Possibilité de créer une page "Dessin" + intégration reconnaissance du Stylet
- "forum" avec messages non instantanée (livre d'or) + la possibilité de créer un chat instantanée entre deux utilisateurs ou groupe d'utilisateurs

V3

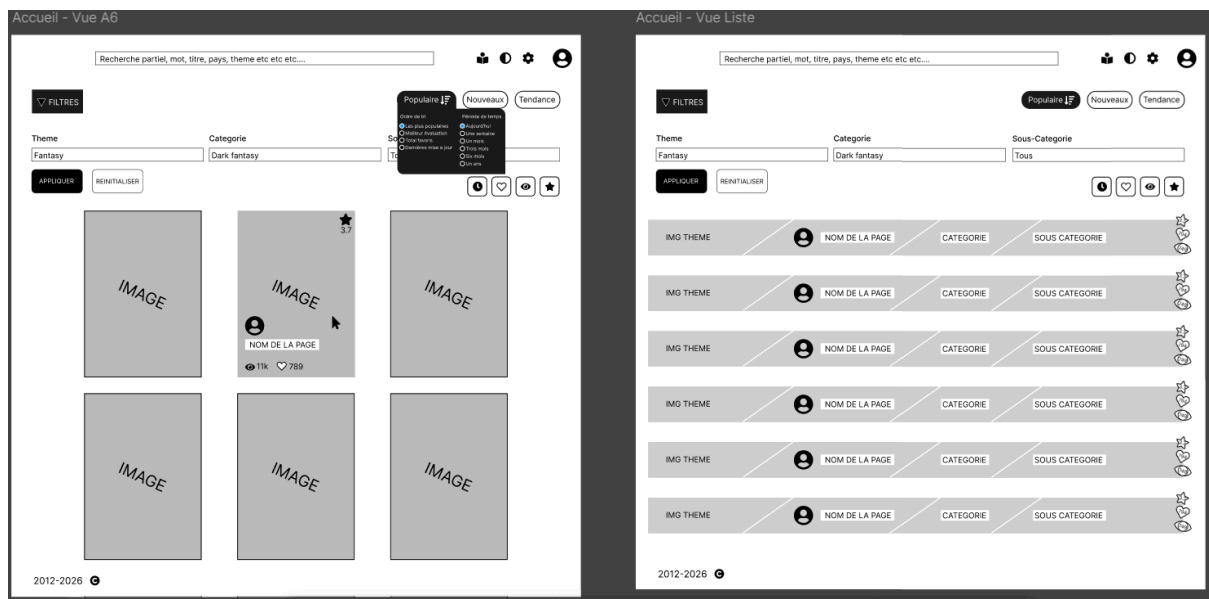
Rendu :

- ajouter la possibilité d'intégrer une carte-monde et définir des points d'intérêt avec annotation qui redirigerai vers des sous-pages ou sous-carte
- intégration mode "18+" avec vérification de l'âge de l'utilisateur, contenu filtré jusqu'à vérification - sauf local
- (Si techniquement possible) Création d'un plugin navigateur qui permettrait de se log avec son compte Utilisateurs, de clic droit sur une image/vidéos ou de sélectionner+clic droit un texte sur n'importe quel site internet et envoyer le résultat dans un onglet "En Vrac" du profil Utilisateur sur le site pour archivage ultérieur ((ou stockage en local + export vers le serveur, après vérification du type / magic bytes / etc))

Conception

Avant de commencer la maquette de mon projet sur Figma j'ai défini les choses dont j'aurai besoins, une interface visuel pour les "pages personnel" en page d'accueil, un système de filtre avancée, un système de CMS drag&drop, un système de traitement de texte...puis j'ai vérifié la disponibilité des librairies pouvant m'aider comme par exemple TipTap, TipTap Resize Image, Vue Color, grid-layout-plus...enfin j'ai chercher des références et commencer ma wireframe puis le MCD, MLD, MPD

Wireframe



Mobile - Filtres





Populaire 

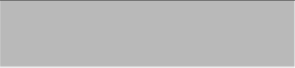
Tendance

New 

Theme

Categorie

Sous-Categorie



2012-2026 

burger - Parametr...





MENU 

Parametres 

Mes Pages

Mes favoris

Mes Favoris

Themes

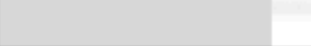
Fiction 

Réel 

Categories

Fantasy 

Nature 



Mobile - Accueil





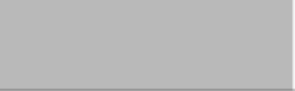

3.7

IMAGE



NOM DE LA PAGE

 11k  789



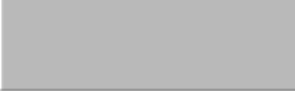
2012-2026 

Mobile - Accueil

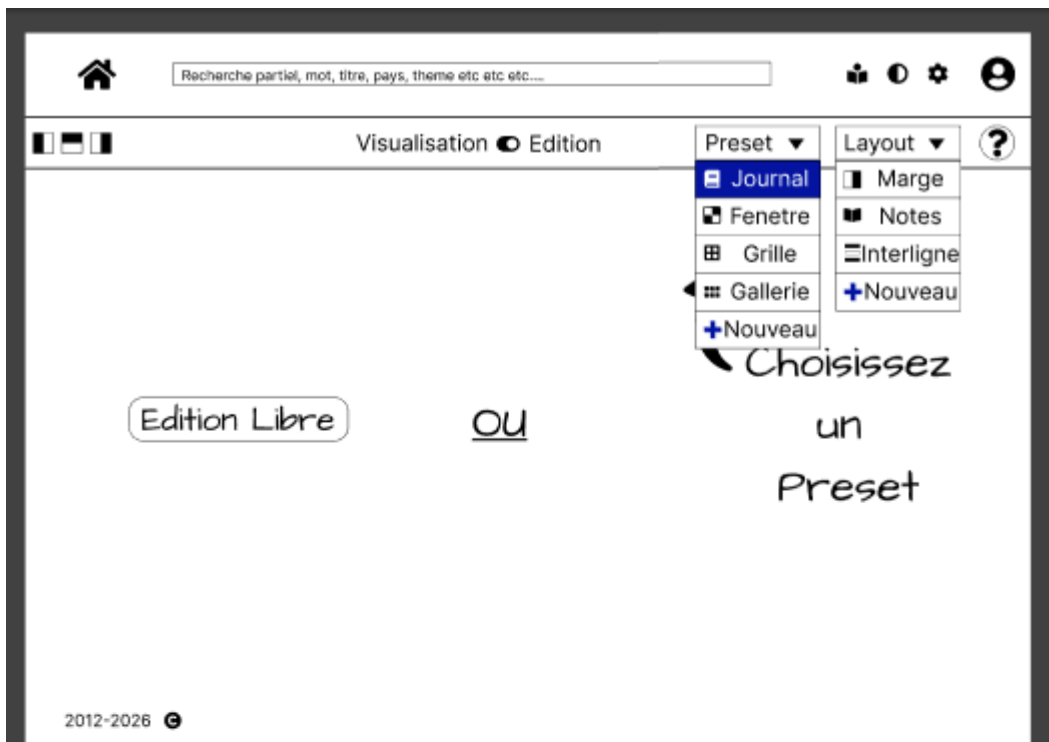




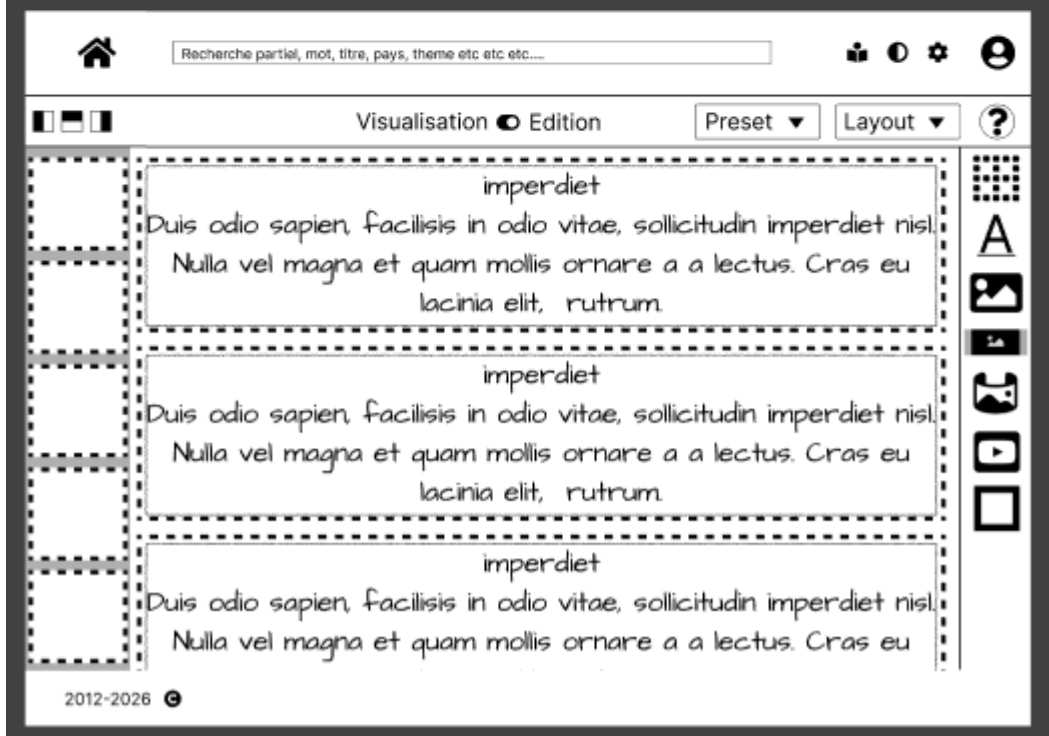
IMAGE

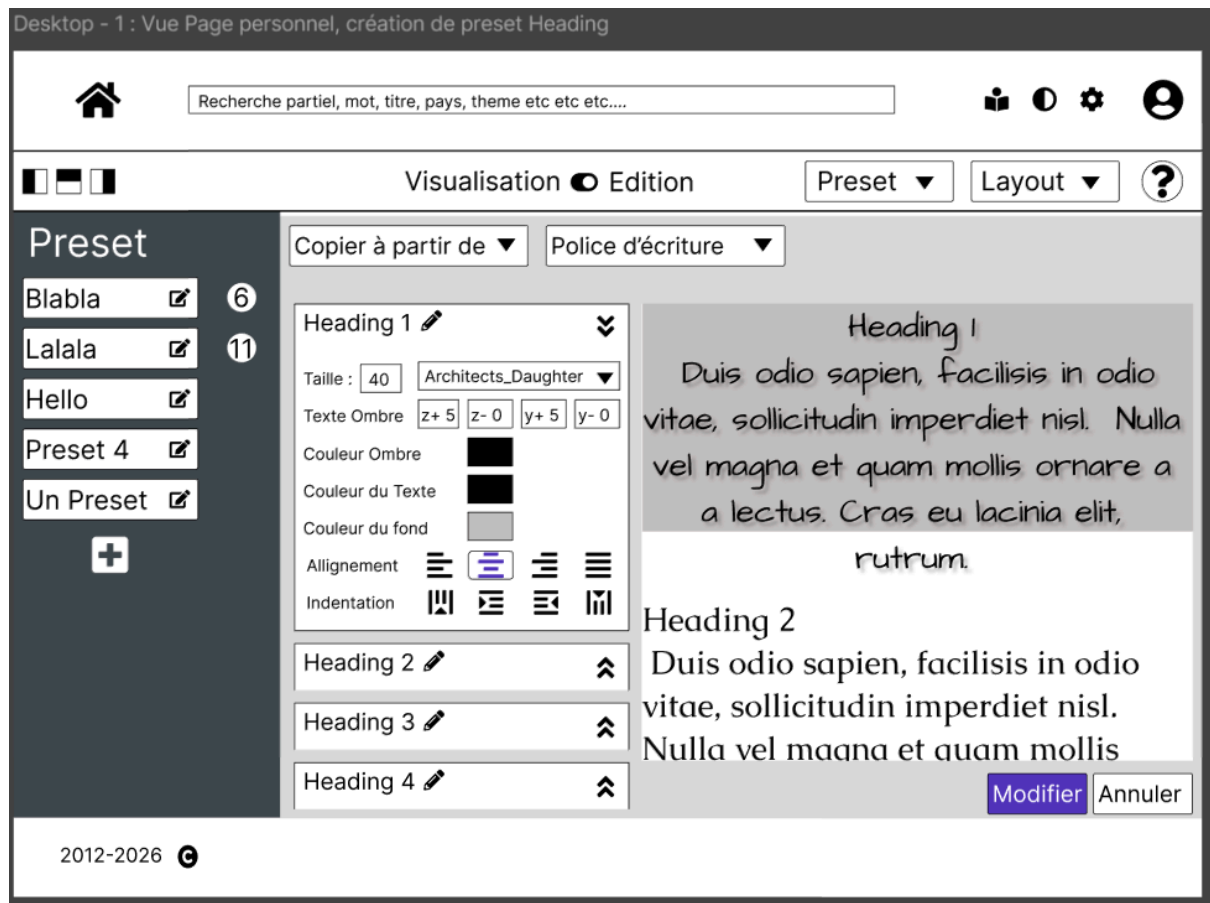


2012-2026 



Desktop - 1 : Vue Page personnel, Preset





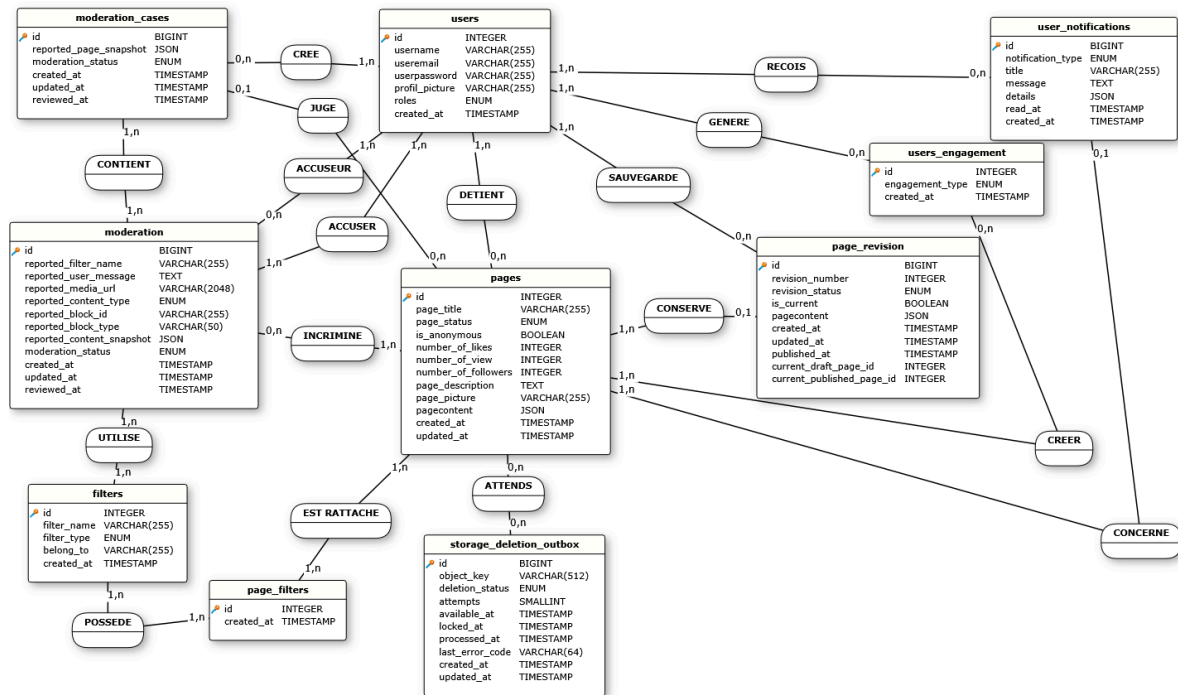
Routes

Route	Description
GET /api	Vérifie que l'API fonctionne et retourne son statut
POST /api/register	Crée un utilisateur
POST /api/login	Authentifie un utilisateur
POST /api/logout	Déconnecte l'utilisateur et détruit sa session
GET /api/me	Retourne le profil de l'utilisateur actuellement connecté.
POST /api/me	Modifie le profil, les identifiants, le mot de passe et ou la photo de l'utilisateur connecté
GET /api/me/notifications	Retourne les notifications de modération
GET /api/me/notifications/unread-count	Retourne uniquement le nombre de notifications non lues
PATCH /api/me/notifications/{id}/read	Marque une notification de l'utilisateur connecté comme lue
GET /api/me/pages	Retourne les pages de l'utilisateur connecté
POST /api/me/pages/{id}/settings	Modifie les paramètres d'une page appartenant à l'utilisateur (titre, statut, filtres...)
GET /api/me/picture?key={key}	Retourne depuis MinIO une photo de profil appartenant à l'utilisateur

Route	Description
GET /api/filters	Retourne les filtres publics et accepte ?type={type} (theme,category,subcategory)
GET /api/pages	Liste les pages publiques avec les paramètres de filtre et de tri facultatifs pages?sort_by=date&sort_order=asc
POST /api/pages	Crée une page initial vide pour l'utilisateur connecté
GET /api/pages/{id}	Retourne le contenu d'une page accessible au visiteurs
GET /api/pages/{id}/draft	Retourne le brouillon d'une page appartenant à l'utilisateur
PUT /api/pages/{id}/draft	Enregistre intégralement le contenu JSON au brouillon
PUT /api/pages/{id}/metadata	Modifie en une opération le titre et les filtres associés à une page
GET /api/pages/{id}/media?key={key}	Retourne un média autorisé depuis MinIO (image, video)
POST /api/pages/{id}/media	Envoi dans MinIO une image ou une vidéo utilisée dans une page lors de l'édition de celle-ci
GET /api/pages/{id}/picture	Retourne l'image de présentation d'une page depuis MinIO
POST /api/pages/{id}/picture	Envoi dans MinIO une image de présentation pour une page
GET /api/pages/{id}/owner-picture	Retourne l'image de profil du propriétaire lorsque la publication n'est pas anonyme
POST /api/pages/{id}/publish	Valide et publie le brouillon d'une page avec son choix d'anonymat
POST /api/pages/{id}/reports	Permet à un utilisateur connecté de signaler une page ou un bloc de contenu
POST /api/links/inspect	Analyse une URL avant son insertion et vérifie son type mime et octets
GET /api/admin/access	Vérifie si le rôle de l'utilisateur connecté est "admin"
GET /api/admin/filters	Retourne tous les filtres
POST /api/admin/filters	Créer un filtre et défini son type et/ou sa relation avec un autre filtre
PATCH /api/admin/filters/{id}	Déplace un filtre dans la hiérarchie ou modifie sa relation
DELETE /api/admin/filters/{id}	Supprime un filtre
GET /api/admin/moderation	Retourne les dossiers et signalements de modération
GET /api/admin/moderation/users	Regroupe les signalements par utilisateurs
GET /api/admin/moderation/cases/{id}/context	Retourne le contenu d'un dossier de modération
POST /api/admin/moderation/cases/{id}/dismiss	Rejette et clôture un dossier de modération
POST /api/admin/moderation/{id}/dismiss	Rejette un signalement individuel
POST /api/admin/moderation/{id}/remove	Retire le contenu signalé et enregistre la décision de modération

MCD - Modèle Conceptuel de Donnée

Schéma réalisé avec l'aide de l'outil "Jmerise"



[illegible]

Dictionnaire de données

Donnée, type et contraintes	Description
users	
id Type : INT NOT NULL, PRIMARY KEY, AUTO INCREMENT	Identifiant unique de l'utilisateur.
username Type : VARCHAR(255) NOT NULL, UNIQUE	Nom public et unique de l'utilisateur.
useremail Type : VARCHAR(255) NOT NULL, UNIQUE	Adresse électronique unique utilisée par l'utilisateur.
userpassword Type : VARCHAR(255) NOT NULL	Empreinte sécurisée du mot de passe de l'utilisateur.
profil_picture Type : VARCHAR(255) NULL	Emplacement de l'image de profil de l'utilisateur.
roles Type : ENUM('user', 'admin') NOT NULL, DEFAULT 'user'	Rôle attribué à l'utilisateur dans l'application.
created_at Type : TIMESTAMP NULL, DEFAULT CURRENT_TIMESTAMP	Date et heure de création du compte utilisateur.
filters	
id Type : INT NOT NULL, PRIMARY KEY, AUTO INCREMENT	Identifiant unique du filtre.
filter_name Type : VARCHAR(255) NOT NULL, UNIQUE	Nom unique affiché pour le filtre.
filter_type Type : ENUM('theme', 'category', 'subcategory', 'moderation') NOT NULL	Nature du filtre.
belong_to Type : INT NULL	Identifiant du filtre parent.
created_at Type : TIMESTAMP NULL, DEFAULT CURRENT_TIMESTAMP	Date et heure de création du filtre.
pages	
id Type : INT NOT NULL, PRIMARY KEY, AUTO INCREMENT	Identifiant unique de la page.
owner_user_id Type : INT NOT NULL, FOREIGN KEY → users.id, ON DELETE CASCADE	Identifiant de l'utilisateur propriétaire de la page.
page_title Type : VARCHAR(255) NOT NULL	Titre de la page.
page_status Type : ENUM('public', 'private', 'banned') NOT NULL, DEFAULT 'private'	État de visibilité ou de modération de la page.
is_anonymous Type : BOOLEAN NOT NULL, DEFAULT FALSE	Indique si l'identité du propriétaire doit être masquée lors de la publication.
number_of_likes Type : INT NULL, DEFAULT 0	Nombre de mentions J'aime reçues par la page.
number_of_view Type : INT NULL, DEFAULT 0	Nombre de consultations de la page.

number_of_followers Type : INT NULL, DEFAULT 0	Nombre d'utilisateurs qui suivent la page.
page_description Type : TEXT NULL	Résumé ou présentation courte de la page.
page_picture Type : VARCHAR(255) NULL	Emplacement de l'image de présentation de la page.
pagecontent Type : JSON NOT NULL	Contenu de la page.
created_at Type : TIMESTAMP NULL, DEFAULT CURRENT_TIMESTAMP	Date et heure de création de la page.
updated_at Type : TIMESTAMP NULL, DEFAULT CURRENT_TIMESTAMP, ON UPDATE CURRENT_TIMESTAMP	Date et heure de la dernière modification de la page.
moderation_cases	
id Type : BIGINT UNSIGNED NOT NULL, PRIMARY KEY, AUTO_INCREMENT	Identifiant unique du dossier de modération.
reported_page_id Type : INT NOT NULL, UNIQUE, FOREIGN KEY → pages.id, ON DELETE CASCADE	Identifiant de la page concernée par le dossier de modération.
reported_page_snapshot Type : JSON NOT NULL	Copie de la page conservée au moment de l'ouverture du dossier.
moderation_status Type : ENUM('pending', 'reviewed', 'dismissed') NOT NULL, DEFAULT 'pending'	État de traitement du dossier de modération.
reviewed_by_user_id Type : INT NULL, FOREIGN KEY → users.id, ON DELETE SET NULL	Identifiant de l'administrateur ayant examiné le dossier.
created_at Type : TIMESTAMP NULL, DEFAULT CURRENT_TIMESTAMP	Date et heure d'ouverture du dossier de modération.
updated_at Type : TIMESTAMP NULL, DEFAULT CURRENT_TIMESTAMP, ON UPDATE CURRENT_TIMESTAMP	Date et heure de la dernière modification du dossier.
reviewed_at Type : TIMESTAMP NULL, DEFAULT NULL	Date et heure auxquelles le dossier a été examiné.
moderation	
id Type : BIGINT UNSIGNED NOT NULL, PRIMARY KEY, AUTO_INCREMENT	Identifiant unique du signalement.
moderation_case_id Type : BIGINT UNSIGNED NOT NULL, FOREIGN KEY → moderation_cases.id, ON DELETE CASCADE	Identifiant du dossier de modération auquel appartient le signalement.
reporter_user_id Type : INT NOT NULL, FOREIGN KEY → users.id, ON DELETE CASCADE, UNIQUE COMPOSITE	Identifiant de l'utilisateur ayant effectué le signalement.
reported_page_id Type : INT NOT NULL, FOREIGN KEY → pages.id, ON DELETE CASCADE, UNIQUE COMPOSITE	Identifiant de la page signalée.
reported_filter_content Type : INT NULL, FOREIGN KEY → filters.id, ON DELETE SET NULL	Identifiant du motif sélectionné pour qualifier le signalement.
reported_filter_name Type : VARCHAR(255) NOT NULL	Nom du motif conservé au moment du signalement.

reported_user_message Type : TEXT NULL	Message complémentaire rédigé par l'auteur du signalement.
reported_media_url Type : VARCHAR(2048) NULL	adresse ou clé Minio du média concernée .
reported_content_type Type : ENUM('page_display', 'page_content') NOT NULL, DEFAULT 'page_display', UNIQUE COMPOSITE, CHECK valid_reported_block	Partie de la page concernée par le signalement.
reported_block_id Type : VARCHAR(255) NOT NULL, DEFAULT chaîne vide, UNIQUE COMPOSITE, CHECK valid_reported_block	Identifiant du bloc signalé, vide lorsque le signalement concerne la présentation générale.
reported_block_type Type : VARCHAR(50) NULL, CHECK valid_reported_block	Nature du bloc de contenu signalé.
reported_content_snapshot Type : JSON NULL	Copie du contenu concerné conservée au moment du signalement.
moderation_status Type : ENUM('pending', 'reviewed', 'dismissed') NOT NULL, DEFAULT 'pending'	État de traitement du signalement.
reviewed_by_user_id Type : INT NULL, FOREIGN KEY → users.id, ON DELETE SET NULL	Identifiant de l'administrateur ayant examiné le signalement.
created_at Type : TIMESTAMP NULL, DEFAULT CURRENT_TIMESTAMP	Date et heure de création du signalement.
updated_at Type : TIMESTAMP NULL, DEFAULT CURRENT_TIMESTAMP, ON UPDATE CURRENT_TIMESTAMP	Date et heure de la dernière modification du signalement.
reviewed_at Type : TIMESTAMP NULL, DEFAULT NULL	Date et heure auxquelles le signalement a été examiné.
page_revision	
id Type : BIGINT UNSIGNED NOT NULL, PRIMARY KEY, AUTO_INCREMENT	Identifiant unique de la révision.
page_id Type : INT NOT NULL, FOREIGN KEY → pages.id, ON DELETE CASCADE, UNIQUE COMPOSITE avec revision_number	Identifiant de la page à laquelle appartient la révision.
created_by_user_id Type : INT NULL, FOREIGN KEY → users.id, ON DELETE SET NULL	Identifiant de l'utilisateur ayant créé ou enregistré la révision.
revision_number Type : INT UNSIGNED NOT NULL, UNIQUE COMPOSITE avec page_id	Numéro chronologique de la révision pour la page.
revision_status Type : ENUM('draft', 'published', 'archived') NOT NULL, DEFAULT 'draft', CHECK valid_current_page_revision	État de la révision.
is_current Type : BOOLEAN NOT NULL, DEFAULT TRUE, CHECK valid_current_page_revision	Indique si la révision est actuellement active.
pagecontent Type : JSON NOT NULL	Contenu structuré enregistré dans la révision.
created_at Type : TIMESTAMP NULL, DEFAULT CURRENT_TIMESTAMP	Date et heure de création de la révision.
updated_at Type : TIMESTAMP NULL, DEFAULT CURRENT_TIMESTAMP, ON UPDATE CURRENT_TIMESTAMP	Date et heure de la dernière modification de la révision.
published_at Type : TIMESTAMP NULL, DEFAULT NULL	Date et heure de publication de la révision.

current_draft_page_id Type : INT NULL, UNIQUE, CHECK valid_current_page_revision	Identifiant de la page lorsque cette révision constitue son brouillon courant.
current_published_page_id Type : INT NULL, UNIQUE, CHECK valid_current_page_revision	Identifiant de la page lorsque cette révision constitue sa version publiée courante.
user_notifications	
id Type : BIGINT UNSIGNED NOT NULL, PRIMARY KEY, AUTO_INCREMENT	Identifiant unique de la notification.
user_id Type : INT NOT NULL, FOREIGN KEY → users.id, ON DELETE CASCADE	Identifiant de l'utilisateur destinataire de la notification.
page_id Type : INT NULL, FOREIGN KEY → pages.id, ON DELETE SET NULL	Identifiant de la page éventuellement concernée par la notification.
notification_type Type : ENUM('moderation_content_removed', 'moderation_page_picture_removed') NOT NULL	Nature de l'événement de modération à l'origine de la notification.
title Type : VARCHAR(255) NOT NULL	Titre affiché pour la notification.
message Type : TEXT NOT NULL	Message principal de la notification.
details Type : JSON NOT NULL	Informations structurées complémentaires associées à la notification.
read_at Type : TIMESTAMP NULL, DEFAULT NULL	Date et heure auxquelles la notification a été lue.
created_at Type : TIMESTAMP NULL, DEFAULT CURRENT_TIMESTAMP	Date et heure de création de la notification.
storage_deletion_outbox	
id Type : BIGINT UNSIGNED NOT NULL, PRIMARY KEY, AUTO_INCREMENT	Identifiant unique de la demande de suppression différée.
page_id Type : INT NULL, FOREIGN KEY → pages.id, ON DELETE SET NULL	Identifiant de la page d'origine ; devient nul si la page est supprimée avant le traitement de la tâche.
object_key Type : VARCHAR(512) NOT NULL, UNIQUE, CHARACTER SET ascii, COLLATE ascii_bin	Clé unique de l'objet à supprimer du stockage.
deletion_status Type : ENUM('pending', 'processing', 'deleted', 'failed') NOT NULL, DEFAULT 'pending'	État d'avancement de la suppression de l'objet.
attempts Type : TINYINT UNSIGNED NOT NULL, DEFAULT 0	Nombre de tentatives déjà effectuées pour supprimer l'objet.
available_at Type : TIMESTAMP NOT NULL, DEFAULT CURRENT_TIMESTAMP	Date et heure à partir desquelles une nouvelle tentative est autorisée.
locked_at Type : TIMESTAMP NULL, DEFAULT NULL	Date et heure de réservation de la tâche par un processus de traitement.
processed_at Type : TIMESTAMP NULL, DEFAULT NULL	Date et heure de traitement définitif de la demande.
last_error_code Type : VARCHAR(64) NULL	Code de la dernière erreur rencontrée pendant la suppression.
created_at Type : TIMESTAMP NULL, DEFAULT CURRENT_TIMESTAMP	Date et heure de création de la demande.
updated_at Type : TIMESTAMP	Date et heure de la dernière mise à jour de la demande.

NULL, DEFAULT CURRENT_TIMESTAMP, ON UPDATE CURRENT_TIMESTAMP	
page_filters	
id Type : INT NOT NULL, PRIMARY KEY, AUTO_INCREMENT	Identifiant unique de l'association entre une page et un filtre.
page_id Type : INT NOT NULL, FOREIGN KEY → pages.id, ON DELETE CASCADE	Identifiant de la page associée au filtre.
filter_id Type : INT NOT NULL, FOREIGN KEY → filters.id, ON DELETE CASCADE	Identifiant du filtre associé à la page.
created_at Type : TIMESTAMP NULL, DEFAULT CURRENT_TIMESTAMP	Date et heure de création de l'association.
users_engagement	
id Type : INT NOT NULL, PRIMARY KEY, AUTO_INCREMENT	Identifiant unique de l'engagement.
user_id Type : INT NOT NULL, FOREIGN KEY → users.id, ON DELETE CASCADE	Identifiant de l'utilisateur à l'origine de l'engagement.
page_id Type : INT NOT NULL, FOREIGN KEY → pages.id, ON DELETE CASCADE	Identifiant de la page concernée par l'engagement.
engagement_type Type : ENUM('like', 'follow') NOT NULL	Nature de l'engagement.
created_at Type : TIMESTAMP NULL, DEFAULT CURRENT_TIMESTAMP	Date et heure de création de l'engagement.

Environnement et Architecture

Environnement de développement

Pour ce projet j'ai travailler sur Windows (OS) j'ai utiliser les outils VScodes (IDE) MySQLWorkbench/PHPmyadmin (GUI bdd) Figma Figjam et draw.io (Conception) Docker (Prod & Nginx) OVH (serveur VPS) et enfin, GitHub et GitKraken

Choix Technique

J'ai choisi une stack PHP côté backend, SQL pour la base de donnée et Vuejs(Nuxtjs) côté frontend - le tout dockérisé afin de faciliter la mise en production en aval en séparant le build prod/dev

Pourquoi?

PHP et Nuxtjs étant donner que je souhaitait faire de la SSG pour le SEO plutôt que le SSR qu'offrait React, chose possible avec uniquement PHP Front/Back néanmoins je souhaitait avoir un framework pour le javascript et j'apprécie la gestion des routes et des composants côté React qu'on retrouve chez Vue/Nuxt

Nuxt génère simplement le SSG via "npm run generate" ce qui génère le HTML en amont, le contenu des pages est ainsi accessible aux moteurs de recherche sans attendre l'exécution

du JavaScript dans le navigateur et donc facilite leur exploration, exemple :

.output/public/login/index.html généré par Nuxt via Nuxt generate – depuis mon container “frontend” dans docker

```
<section
  class="first-background primary-border w-full max-w-md rounded-2xl border-2 p-6 shadow-xl sm:p-8">
  <h1 class="text-center text-3xl font-semibold">Connexion</h1>
  <p class="secondary-color mt-2 text-center text-sm">Retrouvez vos univers et poursuivez leur
    création.</p>
  <form method="post" action="/api/login" class="mt-6 flex flex-col gap-4" novalidate>
    <div><label for="email"
      class="secondary-color block text-sm font-medium">Email</label><input id="email"
      value="" type="email" name="email" required autocomplete="email"
      aria-invalid="false" aria-describedby="login-email-error"
      class="form-control mt-1 block w-full rounded-md border-2 px-3 py-2 shadow-sm focus:outline-n
    </div>
    <div><label for="password" class="secondary-color block text-sm font-medium">Mot de
      passe</label><input id="password" value="" type="password" name="password" required
      autocomplete="current-password" aria-invalid="false"
      class="form-control mt-1 block w-full rounded-md border-2 px-3 py-2 shadow-sm focus:outline-n
      type="button" class="secondary-color mt-2 text-sm underline">Afficher le mot de
      passe</button></div><div><button type="submit"
      class="button-primary mt-2 inline-flex justify-center rounded-md border border-transparent px-4 p
      connecter</button>
    </form>
  <p class="secondary-color mt-5 text-center text-sm">Pas encore de compte ? <a href="/register"
    class="primary-color font-medium underline">S'inscrire</a></p>
</section>
```

On voit que les balises importantes pour le SEO comme `<h1>` `<p>` `<form>` `<section>` sont présentes, mais aussi d'autres non visibles ici, comme `<main>` `<nav>` `<header>` etc etc

`<h1>Connexion</h1>`

`<p>Retrouvez vos univers et poursuivez leur création.</p>`

`<form method="post" action="/api/login" novalidate>`
`<label for="email">Email</label>`
`<input id="email" type="email" name="email">`

`<label for="password">Mot de passe</label>`

`<input id="password" type="password" name="password">`

`<button type="submit">Se connecter</button>`
`</form>`

MySQL 8+ pour la base de données, l'application étant un mix de données integer, varchar mais aussi de données stockées en JSON chose très utile pour les “pages personnel” que comportent mon application celles-ci intégrant un CMS poussée

J'utilise Minio afin de stocker mes données sur le serveur tout en gérant l'archivage des données par utilisateurs selon la catégorie concernée (images, vidéos....) plutôt qu'un bucket S3 chez AWS je l'ai choisi pour sa licence AGPLv3 open-source néanmoins j'ai gardé mon application compatible S3

Enfin ce projet utilise une architecture MVC (Modele, View, Controller) afin de séparer le code entre frontend/backend

```
worldbuilding
|-- backend
|   |-- config
|   |-- controllers
|       |-- admin
|       |-- cms
|       |-- index
|       |-- link-validation
|       |-- minio
|       |-- shared
|-- core
|   |-- brevo
|   |-- cms
|   |-- link-validation
|   |-- minio
|   |-- shared
|-- db
|   |-- migration
|-- models
|   |-- admin
|   |-- cms
|   |-- index
|   |-- minio
|   |-- shared
|-- php
|-- public
|-- routes
|   |-- admin
```

- | -- cms
- | -- index
- | -- link-validation
- | -- shared
- | -- scripts
- | -- minio
- | -- templates
- | -- brevo
- | -- docker
- | -- minio
- | -- mock-data
- | -- Docs
- | -- DossierProjet
- | -- frontend
 - | -- app
 - | -- assets
 - | -- components
 - | -- composables
 - | -- middleware
 - | -- pages
 - | -- plugins
 - | -- types
 - | -- utils
 - | -- views
 - | -- nginx
 - | -- public
- | -- nginx
- | -- secrets

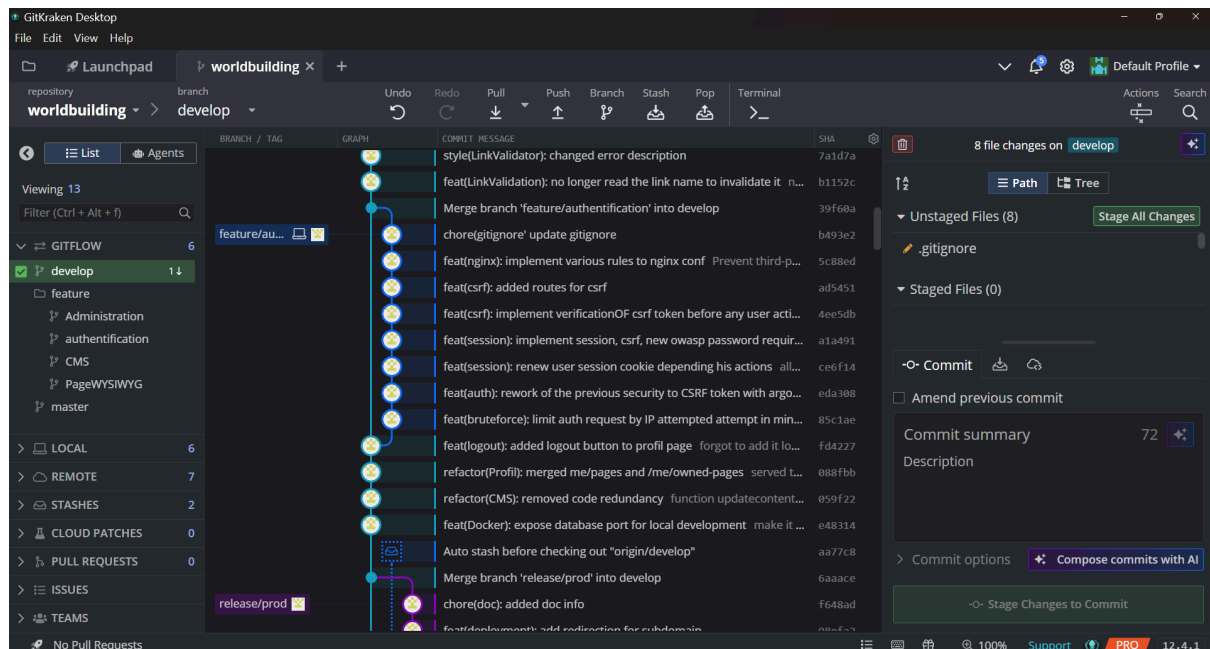
Workflow

Pour ce projet j'ai utilisé GitKraken pour simplifier la gestion des commits, Branch, Merge et résolution d'éventuel conflit, j'ai structuré mes commits selon la convention de nommage "conventional commit" dont voici quelques exemples

TYPE	(SUJET):	COMMENTAIRE
feat	(card):	creation components cards 16:9
fix	(card):	bouton supprimer de nouveau fonctionnel
style	optionnel	indexation prettier
refactor	(auth):	modification d'un code déjà présent
chore	(gitignore):	ajouter .md au .gitignore
fix	(API)!:	changement variable d'environnement

!: "BREAKING CHANGE" comme un changement de clé API qui casse l'application et demande une attention particulière

J'utilise aussi GitFlow permettant une plus simple gestion des branches afin de créer des branches temporaire pour la création ou modification de feature



Afin de sécuriser mon application j'ai ajouter des hook aux commit et lors des push utilisant pre-commit et detect-secrets

```
python -m pip install pre-commit
```

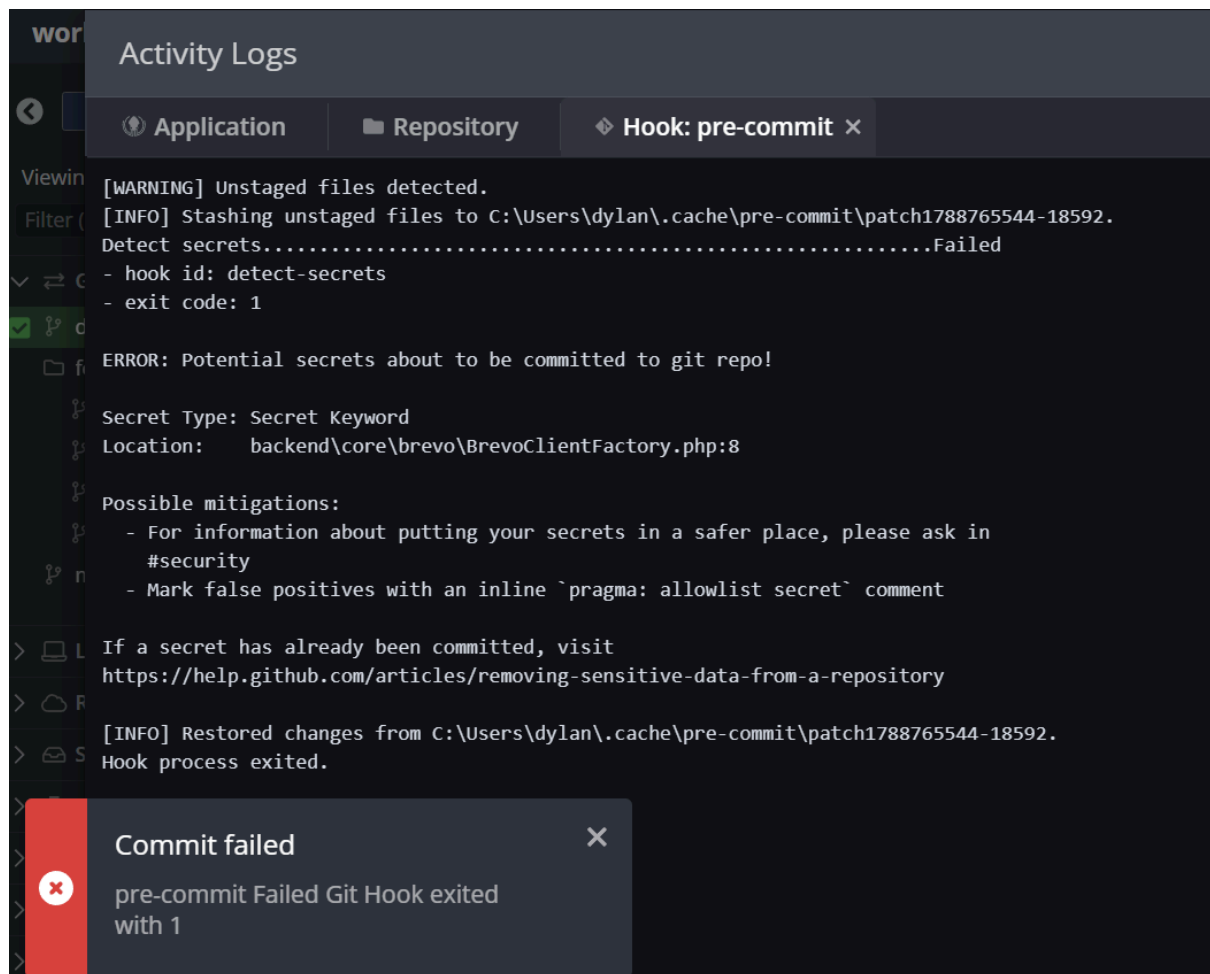
```
python -m pip install detect-secrets
```

```
python -m pre-commit install
```

```
python -m pre-commit install --hook-type pre-push
```

cela me permet de vérifier automatiquement si un .env un secret docker ou une clé API n'est pas protégé lors d'un commit ou d'un push rajoutant une couche de sécurité bien

que cela n'exclut pas de faire attention



Frontend

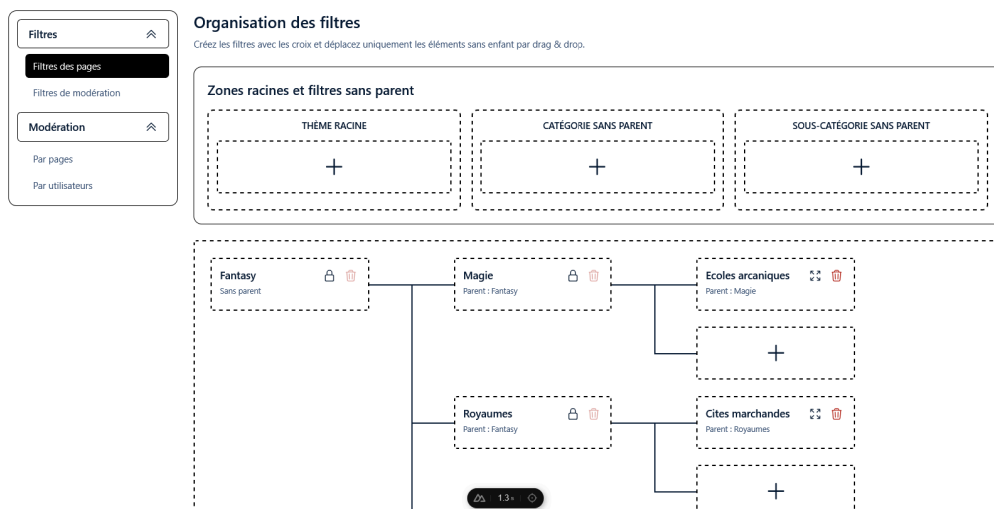
(Site toujours en développement, l'ajout de fonctionnalité est bloqué néanmoins le rendu final pourrait légèrement différer comme les couleurs, boutons, taille...)

formulaire d'inscription, ici les vérifications en temps réel sont faites côté front suivant les différentes règles que comporte le formulaire (limites de caractères, regex....) afin de ne pas frustrer l'utilisateur lors

d'une entrée qui se retrouve refusé lors du POST

The image shows a web application interface for user registration. The page has a dark header with the text 'Déjà un compte ? Se connecter'. The main content area is titled 'Inscription' with the subtitle 'Créez votre compte pour donner vie à vos univers.' Below this, there are three input fields: 'Nom d'utilisateur', 'Email', and 'Mot de passe'. Each field has a validation message: 'Utilisez entre 3 et 50 caractères' for the username, 'Saisissez une adresse email valide' for the email, and 'Utilisez entre 15 et 64 caractères' for the password. There is also a 'Confirmer le mot de passe' field and a 'S'inscrire' button. At the bottom, there is a link 'Déjà un compte ? Se connecter'.

Page administration, gestion des filtres public/modération, ici l'administrateur peut créer de nouveaux filtres et les assigner à un "parent" l'UX a était ma principal préoccupation ici, les éléments ne possédant pas "d'enfants" peuvent être drag&drop afin de permettre une réorganisation rapide des filtres par theme/catégorie/sous catégories



La RGPD imposant la modération du contenu l'interface admin possède aussi un système modération de contenu sur la base de report utilisateurs possible sur chaque champs texte, images, video....créer par un autre utilisateur

ici l'administrateur reçoit les report utilisateurs lié aux pages des utilisateurs, l'administrateur reçoit le sujet concerné, le type de signalement, le message de l'utilisateur ayant signalé ainsi que son

identité — plus bas il reçoit le contenu JSON ainsi que les images/vidéo.... stocké sur Minio, ce contenu JSON a été sauvegardé dans une autre table au moment du report afin de permettre un comparatif contenu report/contenu actuel auquel cas l'utilisateur aurait modifier le contenu entre-temps

Filtres

Filtres des pages

Filtres de modération

Modération

Par pages

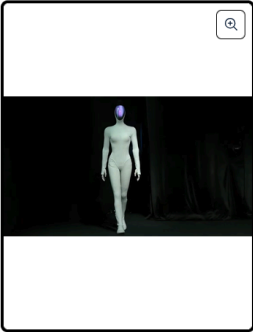
Par utilisateurs

Modération des pages signalées

En attente 1

Traités 1

Rejetés 5



Informations de la page

Robots

Dossier global : En attente

Propriétaire	Statut de publication	Création
 Azerty	public	24 août 2026, 11:44
Dernière modification	Statistiques	
7 sept. 2026, 12:58	0 vues - 0 likes - 0 abonnés	

Ignorer tous les signalements

Page et historique

Contenu JSON de la page

Les blocs ayant un signalement en attente sont rouges. Cliquez sur un bloc pour consulter ses signalements.

JSON signalé

JSON actuel

Filtres

Filtres des pages

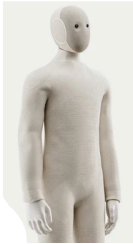
Filtres de modération

Modération

Par pages

Par utilisateurs

Neo



Atlas





Bonjour

ceci est une page exemple de mon CMS



Et juste en dessous, lors ce qu'il "clic" sur l'encadré rouge représentant le contenu signaler par un utilisateur, il vois le contenu signaler en détail avec le type de signalement, le message de la personne ayant signalé ainsi que deux boutons lui permettant de supprimer le contenu concernée ou de juger si cela n'est pas un problème

Filtres

Filtres des pages

Filtres de modération

Modération

Par pages

Par utilisateurs

Signalements du bloc a8f67078-508e-411c-b953-c352b5185732

Mature content - signalé par Azerty

En attente

Test de report par un utilisateur connecté



Ignorer ce signalement

Retirer ce contenu

Historique des révisions

Révision 17 - draft 7 sept. 2026, 12:58 Par Azerty	Révision 16 - published 7 sept. 2026, 12:58 Par Azerty	Révision 15 - archived 7 sept. 2026, 12:58 Par Azerty	Révision 14 - archived 7 sept. 2026, 12:55 Par Azerty	Révision 13 - archived 4 sept. 2026, 09:22 Par Azerty	Révision 12 - archived 3 sept. 2026, 14:32 Par Azerty	Révision 11 - archived 2 sept. 2026, 12:05 Par Azerty
Révision 10 - archived 2 sept. 2026, 12:00 Par Azerty	Révision 9 - archived 2 sept. 2026, 11:54 Par Azerty	Révision 8 - archived 28 août 2026, 13:34 Par Azerty	Révision 7 - archived 28 août 2026, 12:58 Par Azerty	Révision 6 - archived 28 août 2026, 12:36 Par Azerty	Révision 5 - archived 28 août 2026, 12:33 Par Azerty	Révision 4 - archived 28 août 2026, 12:02 Par Azerty
Révision 3 - archived	Révision 2 - archived	Révision 1 - archived				

Modal permettant aux utilisateurs de report un contenu

Rechercher...

Page publique

Neo

Ati

Report content

Nouveau bloc - image

☐ Mature content

☐ Violent or shocking content

☒ Other

Précisez le motif

Spam

☐ Test moderation

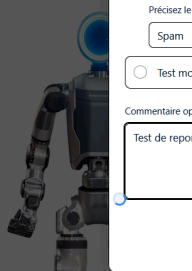
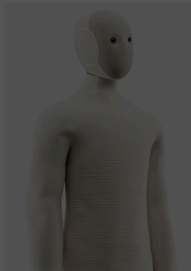
Commentaire optionnel

Test de report par un Utilisateur (sans oublier de screen cette fois...)

72/2000

Annuler

Envoyer le signalement



Sécurité

Hash des mots de passe via Argon2 plutôt que Bcrypt pour sa sécurité concernant les longs mot de passe et capacité de changer le “coût mémoire” de plus Bcrypt étant limité à 72 octets il peut donc ignorer les mots de passes plus long de 15-30 caractere à l'inverse de Argon2

```
public function register(): void
{
    // récupère les données du corps de la requête (POST) et les stocke dans des variables
    $body = Request::body();
    $username = Request::field($body, self::USERNAME_KEYS);
    // convertit l'email en minuscules pour éviter les problèmes de casse lors de la vérification
    $email = strtolower(Request::field($body, self::EMAIL_KEYS));
    $password = PasswordPolicy::normalize(Request::field($body, self::PASSWORD_KEYS, false));

    $this->validateRegister($username, $email, $password);
    // vérifie si le nom d'utilisateur ou l'email existe déjà dans la base de données
    if ($this->users->existsByUsernameOrEmail($username, $email)) {
        Response::error("Nom utilisateur ou email déjà utilise", 409);
    }

    try {
        $user = $this->users->create($username, $email, PasswordPolicy::hash($password));
    } catch (PDOException $exception) {
        // récupère l'erreur SQL et vérifie si c'est une erreur de duplication (code 1062)
        if ((int) ($exception->errorInfo[1] ?? 0) === 1062) {
            Response::error("Nom utilisateur ou email déjà utilise", 409);
        }

        throw $exception;
    }

    $this->sendWelcomeEmail($email);

    $this->respondWithSession("Inscription reussie", $user, 201);
}
```

```

<?php
declare(strict_types=1);

/**
 * Normalizes, validates and hashes passwords used by AuthController and UserProfileController.
 * Plaintext travels only from the request to this class, then an Argon2id hash is persisted by User.
 */
final class PasswordPolicy
{
    public const MIN_LENGTH = 15;
    public const MAX_LENGTH = 64;

    // coût du hachage Argon2
    // recommandation minimum par OWASP: https://owasp.org/www-project-cheat-sheets/cheatsheets/Password_Storage_Cheat_Sheet.html#argon2
    public const ARGON_OPTIONS = [
        "memory_cost" => 19 * 1024,
        "time_cost" => 2,
        "threads" => 1,
    ];

    public static function normalize(string $password): string
    {
        $normalized = self::normalizeForVerification($password);
        $length = mb_strlen($normalized, "UTF-8");

        if ($length < self::MIN_LENGTH || $length > self::MAX_LENGTH) {
            Response::error(
                "Le mot de passe doit contenir entre 15 et 64 caracteres",
                422,
                "invalid_password_length"
            );
        }

        return $normalized;
    }

    public static function normalizeForVerification(string $password): string
    {
        // verifie si les extensions intl et mbstring sont chargées et si la constante PASSWORD_ARGON2ID est définie
        // intl est utilisé pour la normalisation Unicode et mbstring pour la gestion des chaînes multi-octets
        // evite un é ayant un encodage différent de é (e + accent) et donc un mot de passe invalide
        // (U+00E9) vs (U+0065 U+0301)
        // mbstring sert à compter correctement les caractères multi-octets (comme les emojis) pour la vérification de la longueur du mot de passe
        if (!extension_loaded("intl") || !extension_loaded("mbstring") || !defined("PASSWORD_ARGON2ID")) {
            Response::error("Configuration de securite indisponible", 503, "password_security_unavailable");
        }

        if (!mb_check_encoding($password, "UTF-8")) {
            Response::error("Le mot de passe doit etre une chaîne UTF-8 valide", 422, "invalid_password_encoding");
        }

        // assure que é (U+00E9) et é (U+0065 U+0301) sont traités comme le même caractère
        $normalized = Normalizer::normalize($password, Normalizer::FORM_C);

        if (!is_string($normalized)) {
            Response::error("Le mot de passe ne peut pas etre normaliser", 422, "invalid_password_encoding");
        }

        return $normalized;
    }

    public static function hash(string $normalizedPassword): string
    {
        $hash = password_hash($normalizedPassword, PASSWORD_ARGON2ID, self::ARGON_OPTIONS);

        if (!is_string($hash)) {
            throw new RuntimeException("Echec du hachage");
        }

        return $hash;
    }
}

```

☐	Éditer	Copier	Supprimer	5 Azerty	azerty@gmail.com	\$argon2id\$v=19\$m=19456,t=2,p=1\$WGGzalf1Wm1hdEZ3YnZ...	User/5/profilepicture/063a42c927dfd3dac13d811a544...	admin	2026-07-30 08:54:15
☐	Éditer	Copier	Supprimer	14 estatat	test83@gmail.com	\$argon2id\$v=19\$m=19456,t=2,p=1\$eDI2RTgvaQJcGJRdVc...	NULL	user	2026-08-28 09:42:55

Selon les recommandations de OWASP qui recommande de ne pas imposer des caractères spéciaux, de chiffres ou majuscules mais plutôt une taille minimum de 15 caractère et une maximal (protection DDOS) d'autoriser tous les caractères unicode dont les espaces, les symboles complexe ou même les emoji je n'ai pas créer de Regex(MDP) pour la création de

compte

Protection XSS, Vue(Nuxtjs) échappe automatiquement les champs pouvant contenir du Javascript, similaire à la fonction htmlspecialchars de PHP

Validation du type mime et octets lors d'un import vers Minio d'une images, vidéos, audio... par un utilisateur

```
1 reference
private const IMAGE_MIMES = [
    "image/jpeg" => ["extensions" => ["jpg", "jpeg"], "extension" => "jpg"],
    "image/png" => ["extensions" => ["png"], "extension" => "png"],
    "image/webp" => ["extensions" => ["webp"], "extension" => "webp"],
    "image/avif" => ["extensions" => ["avif"], "extension" => "avif"],
    "image/gif" => ["extensions" => ["gif"], "extension" => "gif"],
];
/*
 * limite les extensions de vidéos autoriser à mp4 et webm pour éviter les problèmes de
 * compatibilité avec les navigateurs et les lecteurs vidéo.
 * Les autres formats peuvent être plus difficiles à lire ou à traiter et peuvent également poser des problèmes de sécurité.
 */
1 reference
private const VIDEO_MIMES = [
    "video/mp4" => ["extensions" => ["mp4"], "extension" => "mp4"],
    "video/webm" => ["extensions" => ["webm"], "extension" => "webm"],
];

// sert pour les double extensions comme "image.jpg.php" ou "video.mp4.sh" qui sont dangereuses même si l'extension finale est autorisée
1 reference
private const DANGEROUS_NAME_PARTS = [
    "exe", "cmd", "bat", "com", "msi", "ps1", "sh", "scr", "jar", "php", "phtml", "phar", "js", "vbs",
];
```

```
final class MediaUploadValidator
{
    2 references
    private static function validateSignature(string $path, string $mime): void
    {
        $handle = fopen($path, "rb");

        if ($handle === false) {
            throw new DomainException("Lecture du fichier impossible");
        }

        $header = fread($handle, 32);
        fclose($handle);

        if (!is_string($header)) {
            throw new DomainException("Signature du fichier illisible");
        }

        /*
         * Les signatures binaires sont vérifiées pour éviter les attaques par double extension et les fichiers renommés
         * Les extensions et les types MIME sont déjà validés par validate() avant l'appel de cette méthode
         *
         * fonctionne par liste blanche inversée : si les octets du fichier ne correspondent pas à la signature attendue pour le type MIME détecté, le fichier est rejeté
         * avif et mp4 sont dans des conteneurs ftyp et ISO-BMFF et nécessitent une analyse plus approfondie pour distinguer les marques autorisées des marques génériques
         * ou incompatibles
         */
        $valid = match ($mime) {
            "image/jpeg" => str_starts_with($header, "\xFF\xD8\xFF"),
            "image/png" => str_starts_with($header, "\x89PNG\r\n\x1A\n"),
            "image/gif" => str_starts_with($header, "GIF87a") || str_starts_with($header, "GIF89a"),
            "image/webp" => substr($header, 0, 4) === "RIFF" && substr($header, 8, 4) === "WEBP",
            // AVIF accepte les marques avif ou avis et rejette toute boîte ftyp qui ne les contient pas.
            "image/avif" => self::hasIsoBaseMediaBrand($path, ["avif", "avis"]),
            // MP4 exige une marque MP4 ou AVC explicite et rejette les marques génériques, AVIF, HEIF, 3GP et QuickTime seules.
            "video/mp4" => self::hasIsoBaseMediaBrand($path, ["mp41", "mp42", "avc1"]),
            // WebM exige DocType=webm afin de rejeter Matroska et les autres documents EBML.
            "video/webm" => self::hasEbmldocumentType($path, "webm"),
            default => false,
        };
    }
}
```

Vérification du rôle administrateur de l'utilisateur connecté pour accéder à l'interface administrateur, redirection vers l'index si l'utilisateur tente d'accéder aux routes administrateur sans avoir le rôle nécessaire

```
private function requireAdmin(): int
{
    $userId = Session::userId();

    if ($userId === null || !$this->users->isAdmin($userId)) {
        Response::error("Acces refuse", 403, "access_denied");
    }

    Session::renewForMutation();

    return $userId;
}
```

```
8 references
public function isAdmin(int $id): bool
{
    $statement = $this->pdo->prepare(
        "SELECT 1
        FROM users
        WHERE id = :id AND roles = :role
        LIMIT 1"
    );
    $statement->execute([
        ":id" => $id,
        ":role" => "admin",
    ]);

    return $statement->fetchColumn() !== false;
}
```

par exemple ici un appel sur un GET de la listes des filtres depuis le dashboard admin, la fonction requireAdmin() est systématiquement appeler (voir token CSRF et SESSION ID plus bas) sur chaque actions de l'administrateur

```
public function filters(): void
{
    $this->requireAdmin();

    Response::json(200, [
        "filters" => $this->filters->findAll(),
    ]);
}
```

Backend

Protection des requêtes POST PUT DELETE PATCH via token CSRF générer lors de la création du compte utilisateur, la connexion le tout gérer par une Session PHP partageant la même durée de 7 jours refresh du Token et Session lors d'une interaction sur les NuxtLink de l'application ou lors d'une requête, durée maximum de 30 jours avant re-authentification obligatoire

lors d'une inscription ou connexion réussi la Session est créer avec

```
private function respondWithSession(string $message, array $user, int $status): void
{
    Session::login($user);
    header("Cache-Control: no-store");

    Response::json($status, [
        "message" => $message,
        "user" => $this->publicUser($user),
    ]);
}
```

qui appel login() et lui transmet les informations de l'utilisateur via publicuser()

```
public static function login(array $user): void
{
    self::start();
    session_regenerate_id(true);
    $now = time();
    $_SESSION = [
        "user_id" => (int) $user["id"],
        "created_at" => $now,
        "last_activity" => $now,
        "csrf_token" => self::generateCsrfToken(),
    ];
    self::writeCookie($now + self::IDLE_LIFETIME);
}
```

```
private function publicUser(array $user): array
{
    return [
        "id" => (int) $user["id"],
        "username" => (string) $user["username"],
        "useremail" => (string) $user["useremail"],
        "profil_picture" => $user["profil_picture"],
        "roles" => (string) $user["roles"],
        "created_at" => (string) $user["created_at"],
    ];
}
```

et enfin appel start() qui créer la session côté backend et génère un cookie utilisateur contenant l'id de session avec une durée de validité

```
7 references
private const NAME = "worldbuilding_session";
4 references
private const IDLE_LIFETIME = 604800; // = 7 days
2 references
private const ABSOLUTE_LIFETIME = 2592000; // = 30 days
```

```
public static function start(): void
{
    if (session_status() === PHP_SESSION_ACTIVE) {
        return;
    }

    if (self::isProduction() && !self::isHttps()) {
        Response::error("HTTPS requis", 400, "https_required");
    }

    session_name(self::NAME);
    ini_set("session.use_strict_mode", "1");
    ini_set("session.use_cookies", "1");
    ini_set("session.use_only_cookies", "1");
    ini_set("session.use_trans_sid", "0");
    session_set_cookie_params([
        "lifetime" => self::IDLE_LIFETIME,
        ...self::cookieAttributes(),
    ]);

    if (!session_start()) {
        Response::error("Session indisponible", 503, "session_unavailable");
    }
}
```

Et enfin, on définit les règles du cookie et notamment sa non transmission aux autres sites hors de l'application via samesite => Lax et "httponly" => true pour empêcher une attaque XSS de voler le cookie de l'utilisateur

```
private static function cookieAttributes(): array
{
    return [
        "path" => "/",
        //limite le cookie aux requêtes HTTPS en production
        "secure" => self::isHttps(),
        // empêche l'accès au cookie via JavaScript
        "httponly" => true,
        // empêche l'envoi du cookie pour les requêtes cross-site
        "samesite" => "Lax",
    ];
}
```

Requêtes préparées via PDO pour toutes les requêtes SQL sur le principe du “Ne jamais faire confiance à l'utilisateur” ou encore “Zero Trust”

ici un exemple du formulaire d'inscription route /register
POST=>CONTROLLER=> MODEL

```
$controller = new AuthController($pdo);

if ($route === "/session/activity") {
    $controller->activity();
}

if ($route === "/register") {
    $controller->register();
}

if ($route === "/login") {
    $controller->login();
}

if ($route === "/logout") {
    $controller->logout();
}
```

```

public function register(): void
{
    // récupère les données du corps de la requête (POST) et les stocke dans des variables
    $body = Request::body(self::AUTH_BODY_MAX_BYTES);
    $username = Request::field($body, self::USERNAME_KEYS);
    // convertit l'email en minuscules pour éviter les problèmes de casse lors de la vérification
    $email = strtolower(Request::field($body, self::EMAIL_KEYS));
    $password = Request::field($body, self::PASSWORD_KEYS, false);

    $this->validateRegister($username, $email, $password);
    $password = PasswordPolicy::normalize($password);
    // vérifie si le nom d'utilisateur ou l'email existe déjà dans la base de données
    if ($this->users->existsByUsernameOrEmail($username, $email)) {
        Response::error("Nom utilisateur ou email déjà utilisé", 409);
    }

    try {
        $user = $this->users->create($username, $email, PasswordPolicy::hash($password));
    } catch (PDOException $exception) {
        // récupère l'erreur SQL et vérifie si c'est une erreur de duplication (code 1062)
        if ((int) ($exception->errorInfo[1] ?? 0) === 1062) {
            Response::error("Nom utilisateur ou email déjà utilisé", 409);
        }

        throw $exception;
    }

    $this->sendWelcomeEmail($email);

    $this->respondWithSession("Inscription réussie", $user, 201);
}

```

```

public function create(string $username, string $email, string $passwordHash): array
{
    $statement = $this->pdo->prepare(
        "INSERT INTO users (username, useremail, userpassword)
        VALUES (:username, :useremail, :userpassword)"
    );
    $statement->execute([
        "username" => $username,
        "useremail" => $email,
        "userpassword" => $passwordHash,
    ]);

    $user = $this->findById((int) $this->pdo->lastInsertId());

    if ($user === null) {
        throw new RuntimeException("Utilisateur introuvable après création");
    }

    return $user;
}

```

Utilisateurs unique non ROOT pour la base de donnée,
“WorldBuilding” pour éviter de donner accès à toutes les base
de données à un attaquant – docker compose de production
avec variable d'environnement dans des secrets docker

```
db:
  image: mysql:8
  environment:
    MYSQL_ROOT_PASSWORD_FILE: /run/secrets/mysql_root_password
    MYSQL_DATABASE: "${PROD_MYSQL_DATABASE:-${MYSQL_DATABASE:-worldbuilding}}"
    MYSQL_USER_FILE: /run/secrets/mysql_user
    MYSQL_PASSWORD_FILE: /run/secrets/mysql_password
  secrets:
    - mysql_root_password
    - mysql_user
    - mysql_password
  volumes:
    - db_data:/var/lib/mysql
    - ./backend/db/db.sql:/docker-entrypoint-initdb.d/01-db.sql:ro
  healthcheck:
```

Cela est l'équivalent de la requête SQL suivante :

```
CREATE USER IF NOT EXISTS 'secretadmin'@'%' IDENTIFIED BY
'secretpassword';
GRANT ALL PRIVILEGES ON worldbuilding.* TO 'secretadmin'@'%';
FLUSH PRIVILEGES;
```

Deploiement

Le déploiement a été fait sur un VPS (Virtual Private Server) hébergé
par OVH ainsi qu'un nom de domaine acheté chez OVH, Pour cela j'ai
pris l'offre VPS-1 offrant 4Go de Ram et 40Go de stockage ce qui est
bien suffisant pour mes projets personnel

2027

VPS-1

À partir de
3,81 €

HT/mois
soit 4,57 € TTC/mois

 Configurer

- ✓ 2 vCores
- ✓ 4 Go RAM
- ✓ 40 Go SSD NVMe
- ✓ Sauvegarde automatisée 1 jour
- ✓ Trafic illimité ?
- ✓ 500 Mbit/s bande passante publique

2027

VPS-2

À partir de
7,21 €

HT/mois
soit 8,65 € TTC/mois

 Configurer

- ✓ 4 vCores
- ✓ 8 Go RAM
- ✓ 75 Go SSD NVMe ?
- ✓ Sauvegarde automatisée 1 jour
- ✓ Trafic illimité ?
- ✓ 1 Gbit/s bande passante publique

2027

VPS-3

À partir de
10,40 €

HT/mois
soit 12,48 € TTC/mois

 Configurer

- ✓ 6 vCores
- ✓ 12 Go RAM
- ✓ 100 Go SSD NVMe
- ✓ Sauvegarde automatisée 1 jour
- ✓ Trafic illimité ?
- ✓ 2 Gbit/s bande passante publique

2027

VPS-4

À partir de
19,96 €

HT/mois
soit 23,95 € TTC/mois

 Configurer

- ✓ 8 vCores
- ✓ 24 Go RAM
- ✓ 200 Go SSD NVMe
- ✓ Sauvegarde automatisée 1 jour
- ✓ Trafic illimité ?
- ✓ 3 Gbit/s bande passante publique

Tableau de bord

Bare Metal Cloud

Hosted Private Cloud

Public Cloud

Web Cloud

Télécom

Sumire

Marketplace

Navigation classique

Navigation beta

Découvrez notre nouveau Manager Beta

Français

Dylan Blanc

Commander

Serveurs dédiés

Serveurs Privés Virtuels

Serveurs Privés Virtuels

Managed Bare Metal

Content Delivery Network

Plateformes et services

Logs Data Platform

Network

Stockage et sauvegarde

Licences

Identité, Sécurité & Opérations

Serveurs privés virtuels (VPS) / vps-9bc1a7c2.vps.ovh.net

vps-9bc1a7c2.vps.ovh.net

Accueil

DNS Secondaire

Backup automatisé

Disque additionnel

Bases de données

Monitoring

Votre VPS

Statut

Actif

Nom

vps-9bc1a7c2.vps.ovh.net

Boot

LOCAL

OS / Distribution

Zone

Region OpenStack: os-sbg6

Localisation

Strasbourg (SBG) - France

Votre configuration

Modèle

VPS-1 2027

vCores

2

Ajouter des vCores en passant à la gamme supérieure

Mémoire

4 Go

Ajouter de la mémoire en évoluant vers la gamme supérieure

Stockage

40 Go

Augmenter le stockage en évoluant vers la gamme supérieure

Disques additionnels

Désactivé

Mon offre

Date d'engagement

19 juin 2026

IP

IPv4

51.210.42.110

IPv6

2001:41d0:42b:700::1

Gateway

2001:41d0:42b::1

DNS secondaire

Aucun domaine configuré

Ajoutez un nom de domaine à votre VPS pour partager facilement votre projet avec le monde entier

Ajouter un nom de domaine

Nos guides

Démarrer avec un VPS

Puis je m'y connecte avec une commande SSH via mon terminal intégré à VS code afin de bénéficier de l'interface du navigateur de fichier intégrée, m'y étant déjà connecter précédemment j'ai déjà modifier mon mot de passe et générer une clé SSH avec la commande "ssh-keygen -t ed25519" je possède déjà une clé publique sur mon ordinateur me permettant de me connecter à mon VPS sans avoir besoin d'entrer mon mot de passe à chaque nouvelle connexion

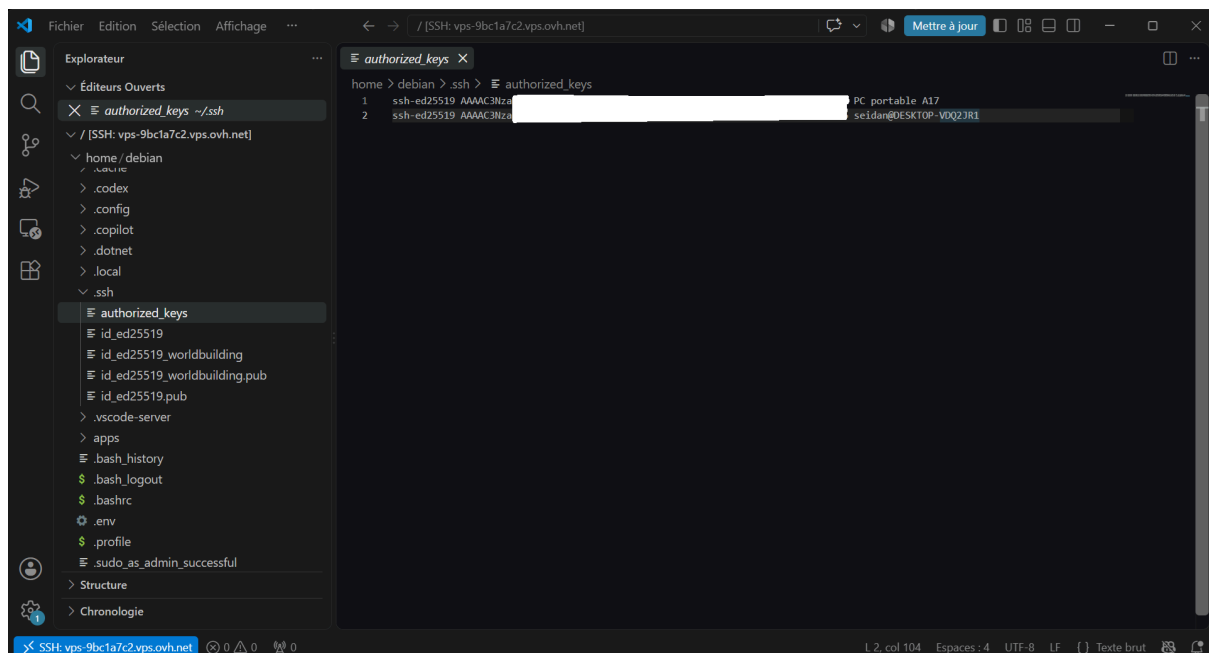
ici la clé ssh publique de mon PC personnel

Nom	Modifié le	Type	Taille			
config	19/06/2026 14:54	Fichier	1 Ko			
id_ed25519	19/06/2026 16:12	Fichier	1 Ko			
id_ed25519.pub	19/06/2026 16:12	Microsoft P...	1 Ko			
known_hosts	19/06/2026 14:45	Fichier	4 Ko			
known_hosts.old	19/06/2026 14:43	Fichier OLD	3 Ko			

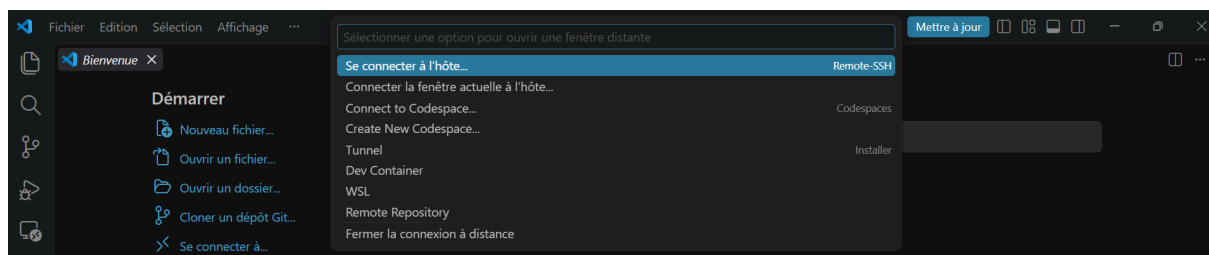
ssh-ed25519 AAAAC3Nza

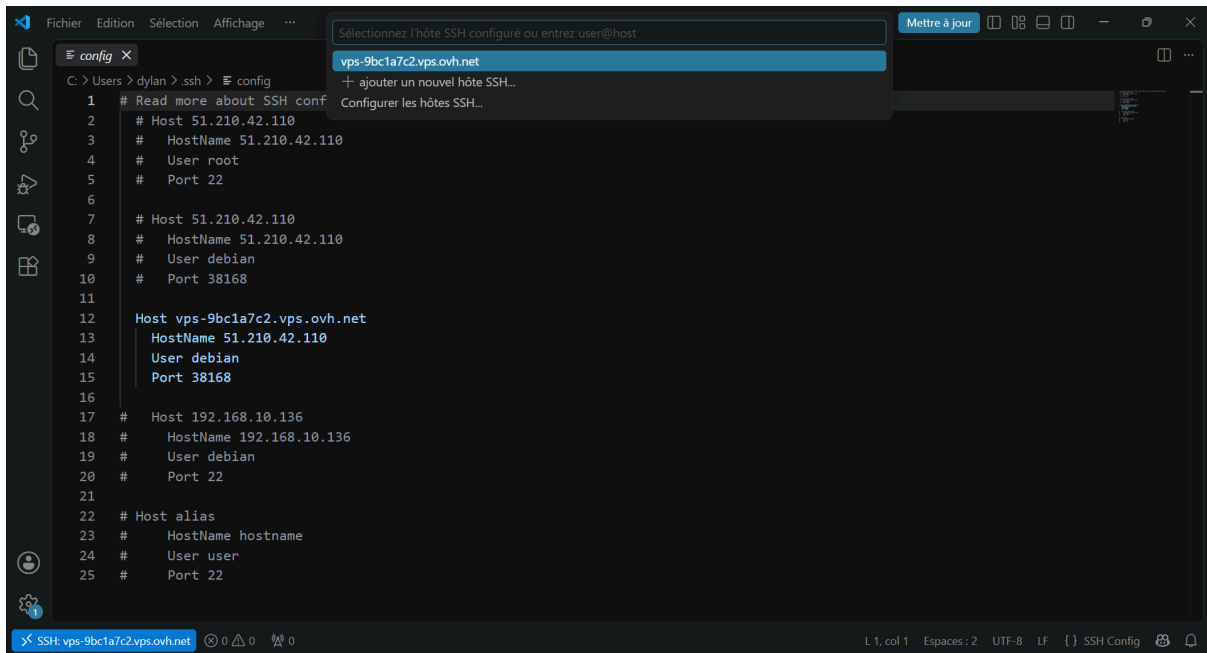
0hm PC portable A17

Et ici la clé autorisée sur mon VPS

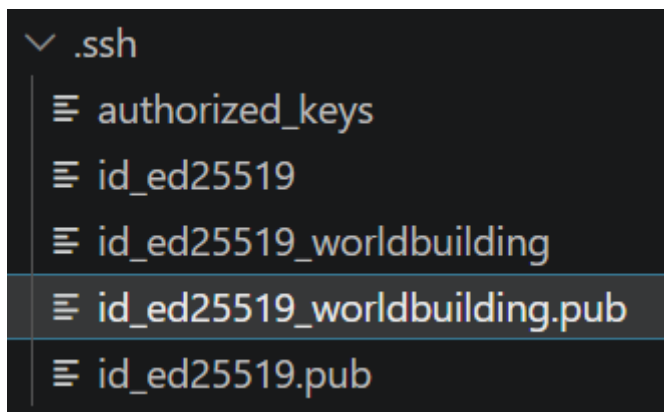


Dans VScode, je configure mon `user@host` permettant la connexion SSH, ici `debian@IPserveurVPS` ayant une clé SSH publique sur mon ordinateur cela me connecte sans demander de mot de passe

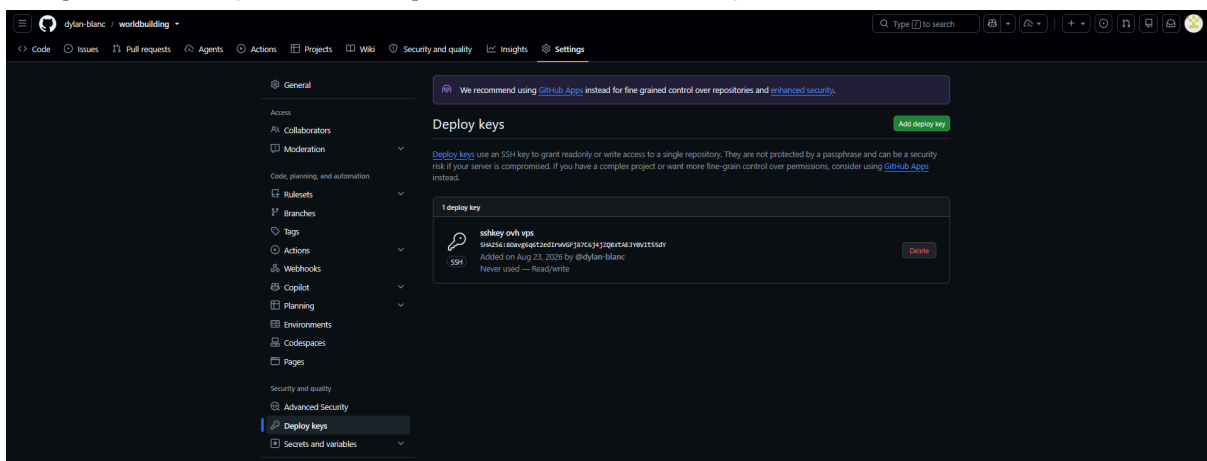




Une fois connecté je créer une clé SSH pour mon repositories GitHub, permettant à mon application hébergée sur mon VPS de communiquer avec GitHub via les commandes Git (ex: git pull, git clone...) avec la commande : `ssh-keygen -t ed25519 -f id_ed25519_worldbuilding -C "worldbuild"`



puis je copie le contenu du fichier `id_ed25519_worldbuilding.pub` (clé publique SSH) dans les options (settings) de mon repositories “worldbuilding”



ANNEXE

MCD originel

